Certificación de Entrega — Entrega 3: Comunicación síncrona y asíncrona

Módulo 3 — Comunicación y sincronización en sistemas distribuidos

> Qué evalúa esta entrega: que el grupo entienda cuándo corresponde cada semántica de comunicación, no solo que el sistema "funcione".

Datos generales

Campo	Valor
Grupo N°	1 (Godoy Cabrera, Santiago Abel · Iriarte Chamorro, Jorge Manuel · Lagoria, Ricardo)
Tag / Release de esta entrega	`v3-comunicacion`
Fecha de entrega	02/11/2026

Checklist de la entrega

Requisito	Cumple	Justificación breve / evidencia
Hay un canal de comunicación síncrono funcionando entre componentes (ej. REST/gRPC).	**Sí**	REST/HTTP en dos tramos: `cliente-web → cliente-api` (login, subida, teselas, metadatos) y `cliente-api → ia-inferencia` (`POST /detectar`: teselas en JPEG, cajas en JSON). El emisor espera la respuesta porque la necesita para seguir. Ver informe §2.
Hay un canal de comunicación asíncrono funcionando entre componentes (ej. cola de mensajes).	**Sí**	Dos usos del mismo patrón de cola en disco con estado consultable: el análisis de un ortomosaico (`POST /imagenes/{id}/analisis` → `202` y `GET` de progreso) y la cola de entrenamientos de `ia-entrenamiento` (API encola, worker consume). Ver informe §3.
Está justificado por qué se usó cada semántica en cada punto del sistema.	**Sí**	Tabla por punto del sistema: lo corto y necesario para continuar es síncrono; lo que dura minutos y no bloquea al usuario es asíncrono. Se explica también por qué no se usó un broker (RabbitMQ/Redis). Ver informe §4.
Se identificó un caso de concurrencia concreto (ej. dos escrituras simultáneas sobre el mismo recurso).	**Sí**	Caso **P1**, ya anticipado en la Entrega 1: dos análisis simultáneos sobre el mismo ortomosaico escriben el mismo `detecciones.json` y gana el último. Ver informe §5.1.
Ese caso de concurrencia está resuelto y la solución puede demostrarse (no solo describirse).	**Sí**	Lock exclusivo por imagen (`flock`) más escritura atómica del JSON: el segundo análisis recibe `409` en vez de pisar al primero. Código en `cliente-api/app/bloqueo.py` y `tareas.py`; demostración con `probar-concurrencia.sh`. Ver informe §5 y el estado de verificación.

Evidencia adicional

Cómo reproducir el caso de concurrencia resuelto (con el entorno levantado y un usuario creado):

```
docker compose -f despliegue/compose.dev.yml up -d --build
USUARIO=RicLagoria CONTRASENA='...' IMAGEN=ruta/al/ortomosaico.tif \
  bash docs/pdc/entrega-3-comunicacion/probar-concurrencia.sh
```

Resultado esperado: un pedido responde `202` y el otro `409`; el análisis aceptado termina en `completado`; `detecciones.json` queda válido y completo. Detalle en el informe.

- Informe descriptivo y técnico: `docs/pdc/entrega-3-comunicacion/informe.md`
- Script de prueba: `probar-concurrencia.sh`
- Código de la solución: `bloqueo.py`, `tareas.py` y los endpoints de `server.py`
- Carpeta en GitHub: https://github.com/RicLagoria/PTI/tree/master/docs/pdc/entrega-3-comunicacion

---

Espacio para el docente

Nota	Corregido por	Fecha de corrección
___ / 10		__ /__ / ____

Observaciones: